

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim Ön Bilgi: YG II TCP soket: `QTcpSocket`, `connectToHost(ip, port)`, `bind/listen`.
Bu hafta HTTP protokolüne bağlanır.

» Ünite 1 — Ağ Katmanları, IP, Port, DNS

TCP/IP Katman Modeli

Dört sorumluluk katmanı, üstten alta: **uygulama** (HTTP, DNS), **taşıma** (TCP/UDP, port burada), **İnternet** (IP, adresleme), **bağlantı** (Ethernet/Wi-Fi).

IP Adresi

Bir ağ arayüzünü tekil tanımlayan sayısal adres (ör. 93.184.216.34); posta adresindeki **bina**. `connectToHost`'taki **ip** argümanı budur.

Port

Aynı IP'deki servisleri ayıran 0–65535 arası sayı; posta adresindeki **daire numarası**. İyi bilinenler: 80 (HTTP), 443 (HTTPS), 22 (SSH).

DNS

Alan adını (`www.example.com`) IP'ye çeviren dağıtık rehber sistemi (santral benzetmesi): tarayıcı → yerel önbellek → çözümleyici → yetkili sunucu.

Ortak Hata / Tuzak

“Port bir *programı* mı belirtir?” Hayır — port, o anda **dinleyen servisi** belirtir; aynı program birden çok porttan dinleyebilir, bir port zamanla farklı programlara geçebilir.

<code>dig +short alan</code>	Alanı IP'ye çözümler (Windows: <code>nslookup</code>)
<code>ping alan</code>	Bağlantı/gecikme (round-trip) kontrolü
<code>curl -v URL</code>	Bağlanılan IP:port'u Connected to satırında gösterir

» Ünite 2 — Standart Web Sunucu Mimarisi

İstemci–Sunucu Modeli

İsteği başlatan taraf (istemci: tarayıcı/`curl`) ile bekleyip cevaplayan taraf (sunucu) rol ayrımı; gişe–müşteri benzetmesi.

Statik İçerik

Sunucunun diskte hazır duran bir dosyayı (HTML, CSS, resim) olduğu gibi gönderdiği durum; her istekte **birebir aynı** cevap (duvardaki afiş).

Dinamik İçerik

Sunucunun isteğe/duruma göre **anında hesapladığı** cevap; her istekte farklı olabilir (gişe memurunun sözlü cevabı). Rastgelelik şart değildir, ayırt edici olan hesaplanma anıdır.

Port 80/443 Sözleşmesi

HTTP varsayılan 80, HTTPS varsayılan 443 portunda dinler — bu bir teknik zorunluluk değil, evrensel bir **sözleşmedir**; URL'de port yoksa tarayıcı bunu dener.

Ters Vekil (Reverse Proxy)

İstemci ile uygulama sunucusu arasına giren, isteği statik/dinamik uygulamaya yönlendiren aracı sunucu (`nginx`); danışma/kapı görevlisi.

Ortak Hata / Tuzak

“Dinamik” = “rastgele/gerçek zamanlı” değildir. Dinamik bir uç nokta bile aynı sonucu verebilir; ayırt edici olan cevabın *hazır dosyadan mı yoksa o anki hesaplamadan mı* geldiğidir.

<code>curl -I URL</code>	Yalnız başlıkları ister (gövdeyi indirmez)
<code>curl -I URL (tekrar)</code>	Statik cevap aynı, dinamik cevap farklı çıkar

» Ünite 3 — WebApp Mimarisi: 3 Katman, API, SPA/SSR

Üç Katmanlı Mimari

Sunum (arayüz, kullanıcı görür) + **iş** (kurallar, ör. “koltuk iki kez satılmaz”) + **veri** (kalıcı saklama); garson/mutfak/kiler benzetmesi. Web'de ayrı süreçlerde çalışır.

API

Bir bileşenin bir başkasına önceden anlaşılmış sözleşmeyle sunduğu arayüz (sipariş fişi benzetmesi); bu ders API'lerde yalnız **JSON** kullanır (Jinja2 anlatılmaz).

SPA vs SSR

SPA: tarayıcı bir kez HTML/JS indirir, sonra yalnız API'den veri çeker (React). **SSR**: her sayfa isteğinde sunucu tam HTML üretir. Bu ders **SPA** izler.

Bu Dersin Yığını

React (Vite, TS) → sunum katmanı; **FastAPI** (Python, Pydantic) → iş katmanı/API; **SQLite** (SQLAlchemy) → veri katmanı.

» Ünite 4 — HTTP İstek/Cevap Anatomisi

HTTP Metotları

GET okur (gövdesiz) · POST yeni kaynak oluşturur (gövdeli) · PUT/PATCH günceller · DELETE siler.

>_ Terminal

```
GET /etkinlikler HTTP/1.1
Host: biletsistemi.example.com
User-Agent: curl/8.4.0
Accept: application/json
```

Beklenen Çıktı

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 52
```

```
{ "etkinlik": "KTU Bahar Senligi", "kalanKoltuk": 42 }
```

Durum Kodu Aileleri

2xx başarılı · 3xx yönlendirme · 4xx istemci hatası (ör. 404) · 5xx sunucu hatası.

<code>curl -v URL</code>	Giden (>) ve gelen (<) tüm başlıkları gösterir
<code>curl -X POST URL -d '{...}'</code>	JSON gövdeli POST isteği gönderir

Ham socket ile elle kurulmuş HTTP GET (YG II `connectToHost` → Python `socket.create_connection`):

```
1 istek = (
2     f"GET / HTTP/1.1\r\n"
3     f"Host: {HOST}\r\n"
4     f"User-Agent: hafta01-socket-istemci\r\n"
5     f"Accept: text/html\r\n"
6     f"Connection: close\r\n"
7     f"\r\n"
8 )
9
10 try:
11     with socket.create_connection((HOST, PORT), timeout=10) as soket:
12         soket.sendall(istek.encode("ascii"))
13         cevap = b""
14         while True:
15             parca = soket.recv(4096)
16             if not parca:
17                 break
18             cevap += parca
19         print(cevap.decode("utf-8", errors="replace"))
20 except OSError as hata:
21     print(
22         f"HATA: {HOST}:{PORT} adresine baglanilamadi ({hata}). "
23         "Internet baglantinizi kontrol edin.",
24         file=sys.stderr,
25     )
26 sys.exit(1)
```

Ortak Hata / Tuzak

`Connection: close` unutulursa `recv(4096)` döngüsü zaman aşımına kadar takılı kalabilir.

5 • İzleme Nerede Kullanılır: GET oku · POST oluştur · PUT/PATCH güncelle · DELETE sil · dig/nslookup: DNS şüphesinde · ping: gecikme şüphesinde · curl -v: terminal incelemesinde · DevTools Ağ sekmesi: görsel inceleme